

Modeling Market Risk

A measure of risk driven by historical data assumes the future will follow the pattern of the past. You need to understand the limitations of that assumption. More importantly, you need to model scenarios in which that pattern breaks down.

— Miles Kennedy

Risk is ubiquitous in finance, but it is hard to quantify. First and foremost, it's important to know how to differentiate the sources of financial risks on the grounds that it might not be a wise move to use the same tools against risks arising from different sources.

Thus, treating the various sources of financial risk differently is crucial because the impacts of those different risks, as well as the tools used to mitigate them, are completely different. Assuming that firms are subject to large market fluctuations, then all assets in their portfolios are susceptible to risk originating from these fluctuations. However, a different tool should be developed to cope with a risk emanating from customer profiles. In addition, keep in mind that different risk factors contribute significantly to asset prices. All of these examples imply that treating risk factors needs careful consideration in finance.

As was briefly discussed previously, these risks are mainly market, credit, liquidity, and operational risks. It is evident that some other types can be added to this list, but they can be thought of as subbranches of these main four risk types, which will be our focus throughout this chapter.

Market risk is the risk arising from changes in financial indicators, such as the exchange rate, interest rate, inflation, and so on. Market risk can be referred to as risk of losses in on- and off-balance-sheet positions arising from movements in market prices (BIS 2020). Let's now see how these factors affect market risk. Suppose that a rise in inflation rates poses a threat to the current profitability of the financial

institutions, since inflation creates pressures on interest rates. This, in turn, affects the cost of funds for borrowers. These instances can be amplified, but we should also note the interactions of these financial risk sources. That is, when a single source of financial risk changes, other risk sources cannot stay constant. Thus to some extent, financial indicators are interrelated, meaning that the interactions of these risk sources should be taken into account.

As you can imagine, there are different tools to manage market risk. Of them, the most prominent and widely accepted tools are value at risk (VaR) and expected short-fall (ES). The ultimate aim of this chapter is to augment these approaches using recent developments in ML. At this juncture, it would be tempting to ask the following questions:

- Do traditional models fail in finance?
- What makes the ML-based model different?

I will start by tackling the first question. The first and foremost challenge that traditional models are unable to address is the complexity of the financial system. Due either to some strong assumptions, or simply their inability to capture the complexity introduced by the data, long-standing traditional models are starting to be replaced by ML-based models.

This fact is well put by Prado (2020):

Considering the complexity of modern financial systems, it is unlikely that a researcher will be able to uncover the ingredients of a theory by visual inspection of the data or by running a few regressions.

To address the second question, it would be wise to think about the working logic of ML models. ML models, as opposed to old statistical methods, try to unveil the associations between variables, identify key variables, and enable us to find out the impact of the variables on the dependent variable without the need for a well-established theory. This is, in fact, the beauty of ML models in the sense that they allow us to discover theories rather than require them:

Many methods from statistics and machine learning (ML) may, in principle, be used for both prediction and inference. However, statistical methods have a long-standing focus on inference, which is achieved through the creation and fitting of a project-specific probability model...

By contrast, ML concentrates on prediction by using general-purpose learning algorithms to find patterns in often rich and unwieldy data.

—Bzdok (2018, p. 232)

In the following section, we'll start our discussion on the market risk models. First, we'll talk about the application of the VaR and ES models. After discussing the traditional application of these models, we will learn how we can improve them by using an ML-based approach. Let's jump in.

Value at Risk (VaR)

The VaR model emerged from a request made by a J.P. Morgan executive who wanted to have a summary report showing possible losses as well as risks that J.P. Morgan was exposed to on a given day. This report would inform executives about the risks assumed by the institution in an aggregated manner. The method by which market risk is computed is known as VaR. This report was the starting point of VaR, and now it has become so widespread that not only institutions prefer using VaR, but its adoption has become required by regulators.

The adoption of VaR dates back to the 1990s, and despite numerous extensions to it and new proposed models, it is still in use. What makes it so appealing? The answer comes from Kevin Dowd (2002, p. 10):

The VaR figure has two important characteristics. The first is that it provides a common consistent measure of risk across different positions and risk factors. It enables us to measure the risk associated with a fixed-income position, say, in a way that is comparable to and consistent with a measure of the risk associated with equity positions. VaR provides us with a common risk yardstick, and this yardstick makes it possible for institutions to manage their risks in new ways that were not possible before. The other characteristic of VaR is that it takes account of the correlations between different risk factors. If two risks offset each other, the VaR allows for this offset and tells us that the overall risk is fairly low.

In fact, VaR addresses one of the most common questions an investor has: *what is the maximum expected loss of my investment?*

VaR provides a very intuitive and practical answer to this question. In this regard, it is used to measure the worst expected loss for a company over a given period and a pre-defined confidence interval. Suppose that a daily VaR of an investment is \$1 million with 95% confidence interval. This would read as there being a 5% chance that an investor might incur a loss greater than \$1 million in a day.

Based on this definition, we can determine that the components of VaR are a confidence interval, a time period, the value of an asset or portfolio, and the standard deviation, as we are talking about risk.

In summary, there are some important points in VaR analysis that need to be highlighted:

- VaR needs an estimation of the probability of loss.
- VaR concentrates on the potential losses. We are not talking about actual or realized losses; rather, VaR is a kind of loss projection.
- VaR has three key ingredients:
 - Standard deviation that defines the level of loss.
 - Fixed time horizon over which risk is assessed.
 - Confidence interval.

VaR can be measured via three different approaches:

- Variance-covariance VaR
- Historical simulation VaR
- Monte Carlo VaR

Variance-Covariance Method

The variance-covariance method is also known as the *parametric* method, because observations are assumed to be normally distributed. The variance-covariance method is commonplace in that returns are deemed to follow normal distribution. The parametric form assumption makes the application of variance-covariance method easy.

As in all VaR approaches, we can either work with a single asset or a portfolio. However, working with a portfolio requires careful treatment in the sense that correlation structure and portfolio variance need to be estimated. At this point, correlation comes into the picture, and historical data is used to calculate correlation, mean, and standard deviation. When augmenting this with an ML-based approach, correlation structure will be our main focus.

Suppose that we have a portfolio consisting of a single asset, as shown in [Figure 1](#). It is shown that the return of this asset is zero and standard deviation is 1, and if the holding period is 1, the corresponding VaR value can be computed from the value of the asset by the corresponding Z-value and standard deviation. Hence, the normality assumption makes things easier, but it is a strong assumption, as there is no guarantee that asset returns are normally distributed; rather, most asset returns do not follow a normal distribution. Moreover, due to the normality assumption, potential risk in tail might not be captured. Therefore the normality assumption comes with a cost. See the following:

```

In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime
import yfinance as yf
from scipy.stats import norm
import requests
from io import StringIO
import seaborn as sns; sns.set()
import warnings
warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (10,6)

In [2]: mean = 0
std_dev = 1
x = np.arange(-5, 5, 0.01)
y = norm.pdf(x, mean, std_dev)
pdf = plt.plot(x, y)
min_ylim, max_ylim = plt.ylim()
plt.text(np.percentile(x, 5), max_ylim * 0.9, '95%:${:.4f}'
        .format(np.percentile(x, 5)))
plt.axvline(np.percentile(x, 5), color='r', linestyle='dashed',
            linewidth=4)
plt.title('Value at Risk Illustration')
plt.show()

In [3]: mean = 0
std_dev = 1
x = np.arange(-5, 5, 0.01)
y = norm.pdf(x, mean, std_dev) ❶
pdf = plt.plot(x, y)
min_ylim, max_ylim = plt.ylim() ❷
plt.text(np.percentile(x, 5), max_ylim * 0.9, '95%:${:.4f}'
        .format(np.percentile(x, 5)))
plt.axvline(np.percentile(x, 5), color='r', linestyle='dashed',
            linewidth=4)
plt.title('Value at Risk Illustration')
plt.show()

```

- ❶ Generating probability density function based on given x, mean, and standard deviation
- ❷ Limiting the x-axis and y-axis

Specifying the location of x at 5% percentile of the x data

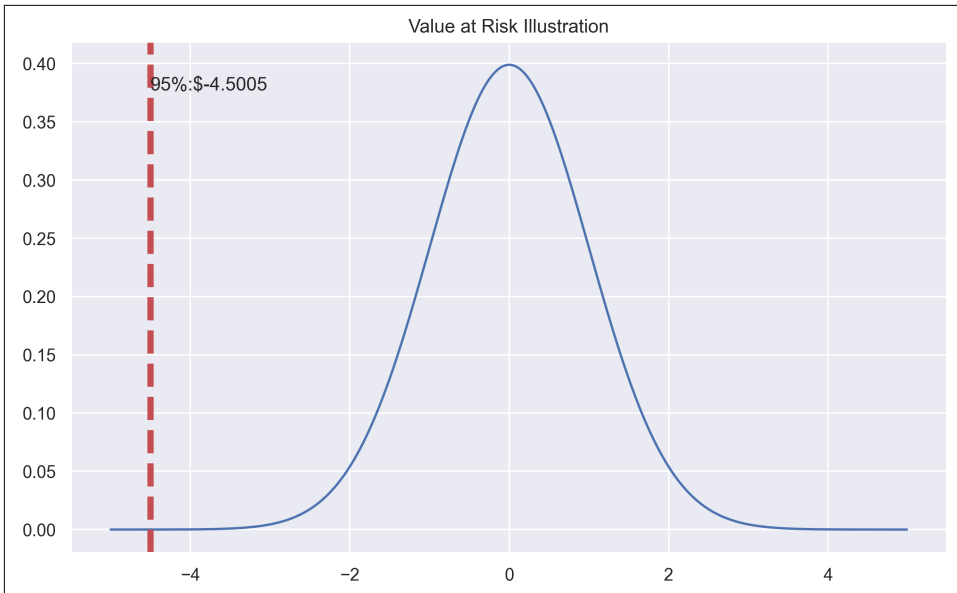


Figure 1. VaR illustration

Following Fama (1965), it was realized that stock price returns do not follow normal distribution due to fat tail and asymmetry. This empirical observation implies that stock returns have higher kurtosis than that of a normal distribution.

Having high kurtosis amounts to fat tail, and this is able to capture the extreme negative returns. As the variance-covariance method is unable to capture fat tail, it cannot, therefore, estimate extreme negative returns that are likely to occur, especially in periods of crisis.

Let's see how we apply the variance-covariance VaR in Python. To illustrate, let's consider a two-asset portfolio. The formula of the variance-covariance VaR is as follows:

$$\text{VaR} = V\sigma_p\sqrt{t}Z_\alpha$$

$$\sigma_p = \sqrt{w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + \rho w_1 w_2 \sigma_1 \sigma_2}$$

$$\sigma_p = \sqrt{w_1\sigma_1 + w_2 + \sigma + 2w_1 w_2 \Sigma_{1,2}}$$

To apply this in code, we start with the following:

```
In [4]: def getDailyData(symbol):
        parameters = {'function': 'TIME_SERIES_DAILY_ADJUSTED',
                      'symbol': symbol,
                      'outputsize': 'full',
                      'datatype': 'csv',
                      'apikey': 'insert your api key here'} ❶

        response = requests.get('https://www.alphavantage.co/query',
                                params=parameters) ❷

        csvText = StringIO(response.text)
        data = pd.read_csv(csvText, index_col='timestamp')
        return data

In [5]: symbols = ["IBM", "MSFT", "INTC"]
        stock3 = []
        for symbol in symbols:
            stock3.append(getDailyData(symbol)[::-1]['close']
                          ['2020-01-01': '2020-12-31'])
        stocks = pd.DataFrame(stock3).T
        stocks.columns = symbols

In [6]: stocks.head()
Out[6]:
```

	IBM	MSFT	INTC
timestamp			
2020-01-02	135.42	160.62	60.84
2020-01-03	134.34	158.62	60.10
2020-01-06	134.10	159.03	59.93
2020-01-07	134.19	157.58	58.93
2020-01-08	135.31	160.09	58.97

- ❶ Identifying the parameters to be used in extracting data from Alpha Vantage
- ❷ Making a request to the Alpha Vantage website

Opening the response file, which is in a text format

Reversing the data that covers the period of 2019-01 to 2019-12 and appending the daily stock prices of IBM, MSFT, and INTC

Alpha Vantage is a data-providing company that partners with major exchanges and institutions. Using Alpha Vantage's API, it is possible to access stock prices with various time intervals (intraday, daily, weekly, and so on), stock fundamentals, and foreign exchange information. For more information, please see [Alpha Vantage's website](#).

We then perform our calculations:

```
In [7]: stocks_returns = (np.log(stocks) - np.log(stocks.shift(1))).dropna() ❶
stocks_returns
Out[7]:
```

	IBM	MSFT	INTC
timestamp			
2020-01-03	-0.008007	-0.012530	-0.012238
2020-01-06	-0.001788	0.002581	-0.002833
2020-01-07	0.000671	-0.009160	-0.016827
2020-01-08	0.008312	0.015803	0.000679
2020-01-09	0.010513	0.012416	0.005580
...	...	...	...
2020-12-24	0.006356	0.007797	0.010679
2020-12-28	0.001042	0.009873	0.000000
2020-12-29	-0.008205	-0.003607	0.048112
2020-12-30	0.004352	-0.011081	-0.013043
2020-12-31	0.012309	0.003333	0.021711

```
[252 rows x 3 columns]

In [8]: stocks_returns_mean = stocks_returns.mean()
weights = np.random.random(len(stocks_returns.columns)) ❷
weights /= np.sum(weights)
cov_var = stocks_returns.cov()
port_std = np.sqrt(weights.T.dot(cov_var).dot(weights)) ❸

In [9]: initial_investment = 1e6
conf_level = 0.95

In [10]: def VaR_parametric(initial_investment, conf_level):
alpha = norm.ppf(1 - conf_level, stocks_returns_mean, port_std) ❹
for i, j in zip(stocks.columns, range(len(stocks.columns))):
    VaR_param = (initial_investment - initial_investment *
                 (1 + alpha))[j] ❺
    print("Parametric VaR result for {} is {} "
          .format(i, VaR_param))
    VaR_param = (initial_investment - initial_investment * (1 + alpha))
    print('---' * 25)
    return VaR_param

In [11]: VaR_param = VaR_parametric(initial_investment, conf_level)
VaR_param
Parametric VaR result for IBM is 42606.16125893139
Parametric VaR result for MSFT is 41024.50194348814
Parametric VaR result for INTC is 43109.25240851776
-----

Out[11]: array([42606.16125893, 41024.50194349, 43109.25240852])
```

❶ Calculating logarithmic return

❷ Drawing random numbers for weights

Generating weights

Calculating covariance matrix

- ⑤ Finding the portfolio standard deviation
- ⑥ Computing the Z-score for a specific value using the percent point function (ppf)
- ⑦ Estimating the variance-covariance VaR model

VaR changes depending on the time horizon in the sense that holding assets for a longer period makes an investor more susceptible to risk. As shown in [Figure 2](#), VaR increases in relation to holding time by the amount of $\sqrt{t}$. Additionally, the holding period is the longest period for portfolio liquidation. Taking into account the reporting purpose, a 30-day period may be a more suitable one for an investor. Therefore, we'll illustrate that period in the following code, in which we generate [Figure 2](#).

```
In [12]: var_horizon = []
        time_horizon = 30
        for j in range(len(stocks_returns.columns)):
            for i in range(1, time_horizon + 1):
                var_horizon.append(VaR_param[j] * np.sqrt(i))
        plt.plot(var_horizon[:time_horizon], "o",
                 c='blue', marker='*', label='IBM')
        plt.plot(var_horizon[time_horizon:time_horizon + 30], "o",
                 c='green', marker='o', label='MSFT')
        plt.plot(var_horizon[time_horizon + 30:time_horizon + 60], "o",
                 c='red', marker='v', label='INTC')
        plt.xlabel("Days")
        plt.ylabel("USD")
        plt.title("VaR over 30-day period")
        plt.legend()
        plt.show()
```

The pros and cons of the variance-covariance method are as follows:

Pros

- Easy to calculate
- Does not require a large number of samples

Cons

- Observations are normally distributed
 - Does not work well with nonlinear structures
 - Requires the computation of the covariance matrix
-

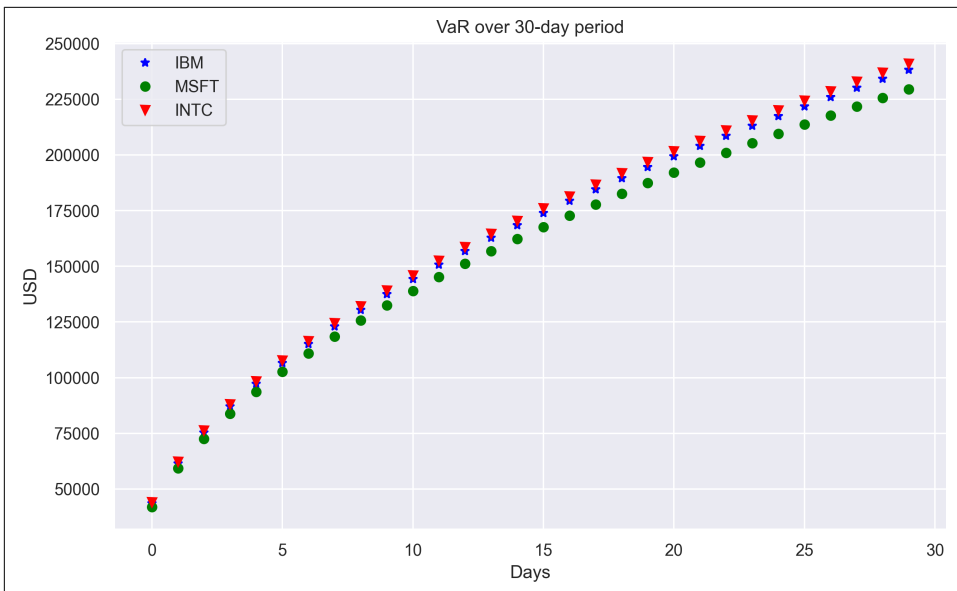


Figure 2. VaR over different horizons

So, even though assuming normality sounds appealing, it may not be the best way to estimate VaR, especially in the case where the asset returns do not have a normal distribution. Luckily, there is another method that does not have a normality assumption, namely the historical simulation VaR model.

The Historical Simulation Method

Having strong assumptions, such as a normal distribution, might be the cause of inaccurate estimations. A solution to this issue is the historical simulation VaR. This is an empirical method: instead of using a parametric approach, we find the percentile, which is the Z-table equivalent of variance-covariance method. Suppose that the confidence interval is 95%; 5% will be used in lieu of the Z-table values, and all we need to do is to multiply this percentile by the initial investment.

The following are the steps taken in the historical simulation VaR:

1. Obtain the asset returns of the portfolio (or individual asset)
2. Find the corresponding return percentile based on confidence interval
3. Multiply this percentile by initial investment

To do this in code, we can define the following function:

```
In [13]: def VaR_historical(initial_investment, conf_level): ❶
         hist_percentile95 = []
```

```

for i, j in zip(stocks_returns.columns,
               range(len(stocks_returns.columns))):
    Hist_percentile95.append(np.percentile(stocks_returns.loc[:, i],
                                           5))
    print("Based on historical values 95% of {}'s return is {:.4f}"
          .format(i, Hist_percentile95[j]))
    VaR_historical = (initial_investment - initial_investment *
                     (1 + Hist_percentile95[j]))
    print("Historical VaR result for {} is {:.2f} "
          .format(i, VaR_historical))
    print('--' * 35)

```

```

In [14]: VaR_historical(initial_investment, conf_level) ❷
Based on historical values 95% of IBM's return is -0.0371
Historical VaR result for IBM is 37081.53
-----
Based on historical values 95% of MSFT's return is -0.0426
Historical VaR result for MSFT is 42583.68
-----
Based on historical values 95% of INTC's return is -0.0425
Historical VaR result for INTC is 42485.39
-----

```

❶ Calculating the 95% percentile of stock returns

❷ Estimating the historical simulation VaR

The historical simulation VaR method implicitly assumes that historical price changes have a similar pattern, i.e., that there is no structural break. The pros and cons of this method are as follows:

Pros

- No distributional assumption
- Works well with nonlinear structures
- Easy to calculate

Cons

- Requires a large sample
- Needs high computing power

The Monte Carlo Simulation VaR

Before delving into the Monte Carlo simulation VaR estimation, it would be good to briefly introduce the Monte Carlo simulation. Monte Carlo is a computerized mathematical method used to make an estimation in cases where there is no closed-form solution, so it is a highly efficient tool for numerical approximation. Monte Carlo relies on repeated random samples from a given distribution.

The logic behind Monte Carlo is well defined by Glasserman (2003, p. 11):

Monte Carlo methods are based on the analogy between probability and volume. The mathematics of measure formalizes the intuitive notion of probability, associating an event with a set of outcomes and defining the probability of the event to be its volume or measure relative to that of a universe of possible outcomes. Monte Carlo uses this identity in reverse, calculating the volume of a set by interpreting the volume as a probability.

From the application standpoint, Monte Carlo is very similar to the historical simulation VaR, but it does not use historical observations. Rather, it generates random samples from a given distribution. Monte Carlo helps decision makers by providing links between possible outcomes and probabilities, which makes it an efficient and applicable tool in finance.

Mathematical Monte Carlo can be defined in the following way:

Let $X_1, X_2, \dots, X_n$ be independent and identically distributed random variables, and $f(x)$ be a real-valued function. The law of large numbers states that:

$$E(f(X)) \approx \frac{1}{N} \sum_{i=1}^N f(X_i)$$

So in a nutshell, a Monte Carlo simulation is doing nothing but generating random samples and calculating their mean. Computationally, it follows these steps:

1. Define the domain
2. Generate random numbers
3. Iterate and aggregate the result

The determination of mathematical π is a simple but illustrative example of Monte Carlo application.

Suppose we have a circle with radius $r = 1$ and an area of 4. The area of a circle is π , and area of a square in which we try to fit the circle is 4. The ratio turns out to be:

$$\frac{\pi}{4}$$

To leave π alone, the proportion between a circle and area can be defined as:

$$\frac{\text{Circumference}_{\text{circle}}}{\text{Area}_{\text{square}}} = \frac{m}{n}$$

Once we equalize these equations, it turns out that:

$$\pi = 4x \frac{m}{n}$$

If we go step by step, the first is to define domain, which is $[-1, 1]$. So the numbers inside the circle satisfy $x^2 + y^2 \leq 1$.

The second step is to generate random numbers to meet this given condition. That is to say, we need to have uniformly distributed random samples, which is a rather easy task in Python. For the sake of practice, I will generate 100 uniformly distributed random numbers using the NumPy library:

```
In [15]: x = np.random.uniform(-1, 1, 100) ❶
         y = np.random.uniform(-1, 1, 100)

In [16]: sample = 100
         def pi_calc(x, y):
             point_inside_circle = 0
             for i in range(sample):
                 if np.sqrt(x[i] ** 2 + y[i] ** 2) <= 1: ❷
                     point_inside_circle += 1
             print('pi value is {}'.format(4 * point_inside_circle/sample))

In [17]: pi_calc(x,y)
         pi value is 3.2

In [18]: x = np.random.uniform(-1, 1, 1000000)
         y = np.random.uniform(-1, 1, 1000000)

In [19]: sample = 1000000

         def pi_calc(x, y):
             point_inside_circle = 0
             for i in range(sample):
                 if np.sqrt(x[i] ** 2 + y[i] ** 2) < 1:
                     point_inside_circle += 1
             print('pi value is {:.2f}'.format(4 * point_inside_circle/sample))

In [20]: pi_calc(x,y)
         pi value is 3.14

In [21]: sim_data = pd.DataFrame([])
         num_reps = 1000
         n = 100
         for i in range(len(stocks.columns)):
             mean = np.random.randn(n).mean()
             std = np.random.randn(n).std()
             temp = pd.DataFrame(np.random.normal(mean, std, num_reps))
             sim_data = pd.concat([sim_data, temp], axis=1)
         sim_data.columns = ['Simulation 1', 'Simulation 2', 'Simulation 3']
```

```

In [22]: sim_data
Out[22]:      Simulation 1 Simulation 2 Simulation 3
0         1.587297    -0.256668    1.137718
1         0.053628    -0.177641   -1.642747
2        -1.636260    -0.626633    0.393466
3         1.088207    0.847237    0.453473
4        -0.479977   -0.114377   -2.108050
..         ...         ...         ...
995        1.615190    0.940931    0.172129
996       -0.015111   -1.149821   -0.279746
997       -0.806576   -0.141932   -1.246538
998        1.609327    0.582967   -1.879237
999       -0.943749   -0.286847    0.777052

[1000 rows x 3 columns]

In [23]: def MC_VaR(initial_investment, conf_level):
MC_percentile95 = []
for i, j in zip(sim_data.columns, range(len(sim_data.columns))):
    MC_percentile95.append(np.percentile(sim_data.loc[:, i], 5))
    print("Based on simulation 95% of {}'s return is {:.4f}"
          .format(i, MC_percentile95[j]))
    VaR_MC = (initial_investment - initial_investment *
              (1 + MC_percentile95[j]))
    print("Simulation VaR result for {} is {:.2f} "
          .format(i, VaR_MC))
    print('---' * 35)

In [24]: MC_VaR(initial_investment, conf_level)
Based on simulation 95% of Simulation 1's return is -1.7880
Simulation VaR result for Simulation 1 is 1787990.69
-----
Based on simulation 95% of Simulation 2's return is -1.6290
Simulation VaR result for Simulation 2 is 1628976.68
-----
Based on simulation 95% of Simulation 3's return is -1.5156
Simulation VaR result for Simulation 3 is 1515623.93
-----

```

- ❶ Generating random numbers from uniform distribution
- ❷ Checking if points are inside the circle, which has a radius of 1

Calculating 95% of every stock return and appending the result in the list named MC_percentile95

Estimating Monte Carlo VaR

VaR is not a **coherent risk** measure, which requires satisfying certain conditions or axioms. You can think of these axioms as technical requirements for a risk measure.

Here are the four axioms of a coherent risk measure:

Translation invariance

For all outcomes Y and a constant $a \in \mathbb{R}$, we have

$$VaR(Y + a) = VaR(Y) + a$$

which means that if a riskless amount a is added to the portfolio, it results in lowering VaR by a .

Subadditivity

For all Y_1 and Y_2 , we have

$$VaR(Y_1 + Y_2) \leq VaR(Y_1) + VaR(Y_2)$$

This axiom stresses the importance of diversification in risk management. Take Y_1 and Y_2 as two assets: if they are both included in the portfolio, then that results in lower VaR than having them separately. Let's check whether VaR satisfies the subadditivity assumption:

```
In [43]: asset1 = [-0.5, 0, 0.1, 0.4] ❶
         VaR1 = np.percentile(asset1, 90)
         print('VaR for the Asset 1 is {:.4f}'.format(VaR1))
         asset2 = [0, -0.5, 0.01, 0.4] ❷
         VaR2 = np.percentile(asset2, 90)
         print('VaR for the Asset 2 is {:.4f}'.format(VaR2))
         VaR_all = np.percentile(asset1 + asset2, 90)
         print('VaR for the portfolio is {:.4f}'.format(VaR_all))
         VaR for the Asset 1 is 0.3100
         VaR for the Asset 2 is 0.2830
         VaR for the portfolio is 0.4000

In [44]: asset1 = [-0.5, 0, 0.05, 0.03] ❶
         VaR1 = np.percentile(asset1, 90)
         print('VaR for the Asset 1 is {:.4f}'.format(VaR1))
         asset2 = [0, -0.5, 0.02, 0.8] ❷
         VaR2 = np.percentile(asset2, 90)
         print('VaR for the Asset 2 is {:.4f}'.format(VaR2))
         VaR_all = np.percentile(asset1 + asset2, 90)
         print('VaR for the portfolio is {:.4f}'.format(VaR_all))
         VaR for the Asset 1 is 0.0440
         VaR for the Asset 2 is 0.5660
         VaR for the portfolio is 0.2750
```

❶ Asset return for the first asset

❷ Asset return for the second asset

It turns out that portfolio VaR is less than the sum of the individual VaRs, which makes no sense due to the risk mitigation through diversification. More elaborately, portfolio VaR should be lower than the sum of individual VaRs via diversification, as diversification mitigates risk, which in turn reduces the portfolio VaR.

Positive homogeneity

For all outcomes Y and $a > 0$, we have

$$VaR(aY) = aVaR(Y)$$

which implies that the risk and value of the portfolio go in tandem—that is, if the value of a portfolio increases by an amount a , the risk goes up by a .

Monotonicity

For any two outcomes, Y_1 and Y_2 , if $Y_1 \leq Y_2$, then:

$$VaR(Y_2) \leq VaR(Y_1)$$

At first, this may seem puzzling, but it is intuitive in the sense that monotonicity implies a lower VaR in the case of higher asset returns.

We now know that VaR is not a coherent risk measure. However, VaR is not the only tool by which we estimate market risk. Expected shortfall is another, and coherent, market risk measure.

Expected Shortfall

Unlike VaR, ES focuses on the tail of the distribution. More specifically, ES enables us to take into account unexpected risks in the market. However, this doesn't mean that ES and VaR are two entirely different concepts. Rather, they are related—that is, it is possible to express ES *using* VaR.

Let's assume that loss distribution is continuous; then ES can be mathematically defined as:

$$ES_{\alpha} = \frac{1}{1-\alpha} \int_{\alpha}^1 q_u du$$

where q denotes the quantile of the loss distribution. The ES formula suggests that it is nothing but a probability weighted average of $(1 - \alpha)$ % of losses.

Let's substitute q_u and VaR, which gives us the following equation:

$$ES_{\alpha} = \frac{1}{1-\alpha} \int_{\alpha}^1 VaR_u du$$

Alternatively, it is the mean of losses exceeding VaR:

$$ES_{\alpha} = E(L | L > VaR_{\alpha})$$

Loss distribution can be continuous or discrete and, as you can imagine, if it takes the discrete form, the ES is different such that

$$ES_{\alpha} = \frac{1}{1-\alpha} \sum_{n=0}^1 \max(L_n) \Pr(L_n)$$

where $\max(L_n)$ shows the highest n^{th} loss, and $\Pr(L_n)$ indicates probability of n^{th} highest loss. In code, we can formulate this as:

```
In [45]: def ES_parametric(initial_investment , conf_level):
        alpha = - norm.ppf(1 - conf_level,stocks_returns_mean,port_std)
        for i, j in zip(stocks.columns, range(len(stocks.columns))):
            VaR_param = (initial_investment * alpha)[j] ❶
            ES_param = (1 / (1 - conf_level)) \
                * initial_investment \
                * norm.expect(lambda x: x,
                    lb = norm.ppf(conf_level,
                        stocks_returns_mean[j],
                        port_std),
                    loc = stocks_returns_mean[j],
                    scale = port_std) ❷
            print(f"Parametric ES result for {i} is {ES_param}")

In [46]: ES_parametric(initial_investment, conf_level)
Parametric ES result for IBM is 52776.42396231898
Parametric ES result for MSFT is 54358.083277762125
Parametric ES result for INTC is 52273.33281273264
```

❶ Estimating the variance-covariance VaR

❷ Given the confidence interval, estimating the ES based on VaR

ES can also be computed based on the historical observations. Like the historical simulation VaR method, parametric assumption can be relaxed. To do that, the first return (or loss) corresponding to the 95% is found, and then the mean of the observations greater than the 95% gives us the result.

Here is what we do in code:

```
In [47]: def ES_historical(initial_investment, conf_level):
        for i, j in zip(stocks_returns.columns,
            range(len(stocks_returns.columns))):
            ES_hist_percentile95 = np.percentile(stocks_returns.loc[:, i],
                5) ❶
            ES_historical = stocks_returns[str(i)][stocks_returns[str(i)] >=
```

```

                                ES_hist_percentile95]\
                                .mean() ❷
print("Historical ES result for {} is {:.4f} "
      .format(i, initial_investment * ES_historical))

In [48]: ES_historical(initial_investment, conf_level)
Historical ES result for IBM is -64802.3898
Historical ES result for MSFT is -65765.0848
Historical ES result for INTC is -88462.7404

```

- ❶ Calculating the 95% of the returns
- ❷ Estimating the ES based on the historical observations

Thus far, we have seen how to model the expected shortfall in a traditional way. Now, it is time to introduce an ML-based approach to further enhance the estimation performance and reliability of the ES model.
